

▼ Column Info

Data fields

Most of the fields are self-explanatory. The following are descriptions for those that aren't.

Id - an Id that represents a (Store, Date) tuple within the test set

Store - a unique Id for each store

Sales - the turnover for any given day (this is what you are predicting)

Customers - the number of customers on a given day

Open - an indicator for whether the store was open: 0 = closed, 1 = open

StateHoliday - indicates a state holiday. Normally all stores, with few exceptions, are closed on state holidays. Note that

SchoolHoliday - indicates if the (Store, Date) was affected by the closure of public schools

StoreType - differentiates between 4 different store models: a, b, c, d

Assortment - describes an assortment level: a = basic, b = extra, c = extended

CompetitionDistance - distance in meters to the nearest competitor store

CompetitionOpenSince[Month/Year] - gives the approximate year and month of the time the nearest competitor was opened

Promo - indicates whether a store is running a promo on that day

Promo2 - Promo2 is a continuing and consecutive promotion for some stores: 0 = store is not participating, 1 = store is part

Promo2Since[Year/Week] - describes the year and calendar week when the store started participating in Promo2

PromoInterval - describes the consecutive intervals Promo2 is started, naming the months the promotion is started anew. E.g.

▼ Module & Data Loading

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
import kagglehub
path = kagglehub.dataset_download("pratyushakar/rossmann-store-sales")
```

Using Colab cache for faster access to the 'rossmann-store-sales' dataset.

```
import os
os.listdir(path)

['store.csv', 'train.csv', 'test.csv']
```

```
df=pd.read_csv(path+'/train.csv')
df.head()
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	
0	1	5	2015-07-31	5263	555	1	1	0	1	
1	2	5	2015-07-31	6064	625	1	1	0	1	
2	3	5	2015-07-31	8314	821	1	1	0	1	

```
stores=pd.read_csv(path+'/store.csv')
stores.head()
```

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	CompetitionOpenSinceYear	Promo2	Promo2Sin
0	1	c	a	1270.0	9.0	2008.0	0	
1	2	a	a	570.0	11.0	2007.0	1	
2	3	a	a	14130.0	12.0	2006.0	1	
3	4	c	c	620.0	9.0	2009.0	0	
4	5	a	a	29910.0	4.0	2015.0	0	

Next steps:

[Generate code with stores](#)

[New interactive sheet](#)

▼ Data Cleaning

> Null Values

↳ 1 cell hidden

> Duplicated

↳ 1 cell hidden

> Data Types

↳ 3 cells hidden

> Outliers

↳ 1 cell hidden

▼ EDA

▼ Feature Engineering Maybe

Extracting Date Values

```
df['Day_name'] = df['Date'].dt.day_name()
```

```
df['Month_name'] = df['Date'].dt.month_name()
df['Month'] = df['Date'].dt.month
df['Year'] = df['Date'].dt.year
```

Holiday Standardization

```
df['StateHoliday'].unique()
```

```
array(['0', 'a', 'b', 'c', 0], dtype=object)
```

```
df['StateHoliday']=df['StateHoliday'].map({0:0, '0':0,'a':1,'b':1,'c':1})
```

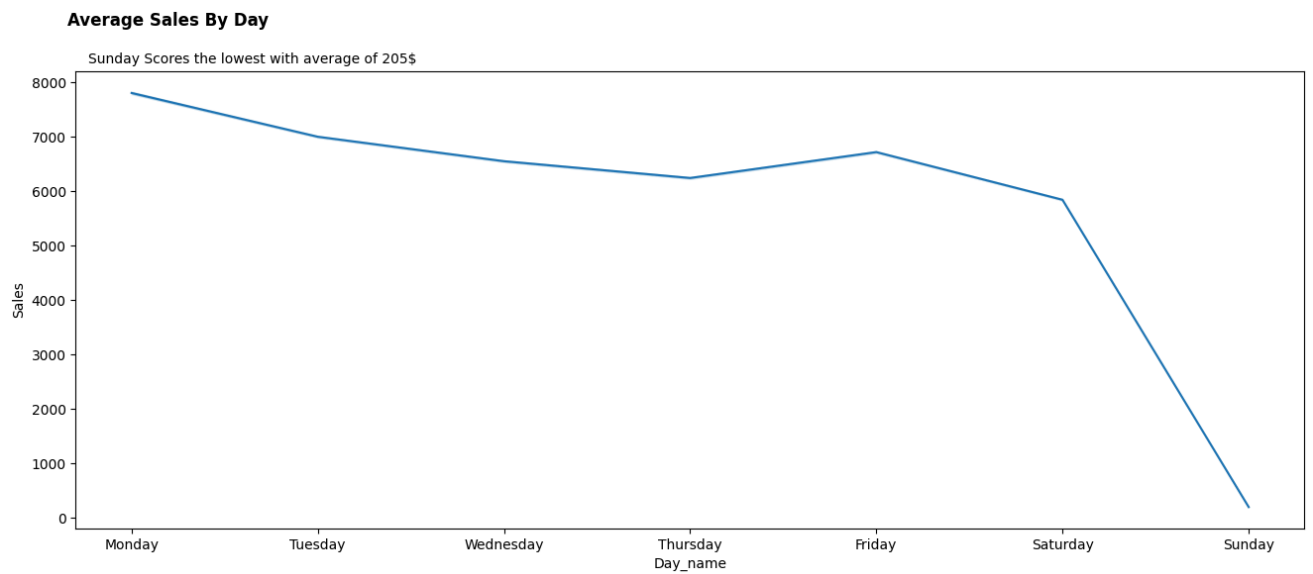
```
df['StateHoliday'].unique()
```

```
array([0, 1])
```

✓ Sales By Day

✓ Plotting Day Data

```
plt.figure(figsize=(16,6))
sns.lineplot(data=df.sort_values(by='DayOfWeek'),x='Day_name',y='Sales')
plt.suptitle('Average Sales By Day', fontweight='bold', x=0.12, fontsize=12,ha='left')
plt.title('Sunday Scores the lowest with average of 205$', x=0.01, fontsize=10,ha='left')
plt.show()
```

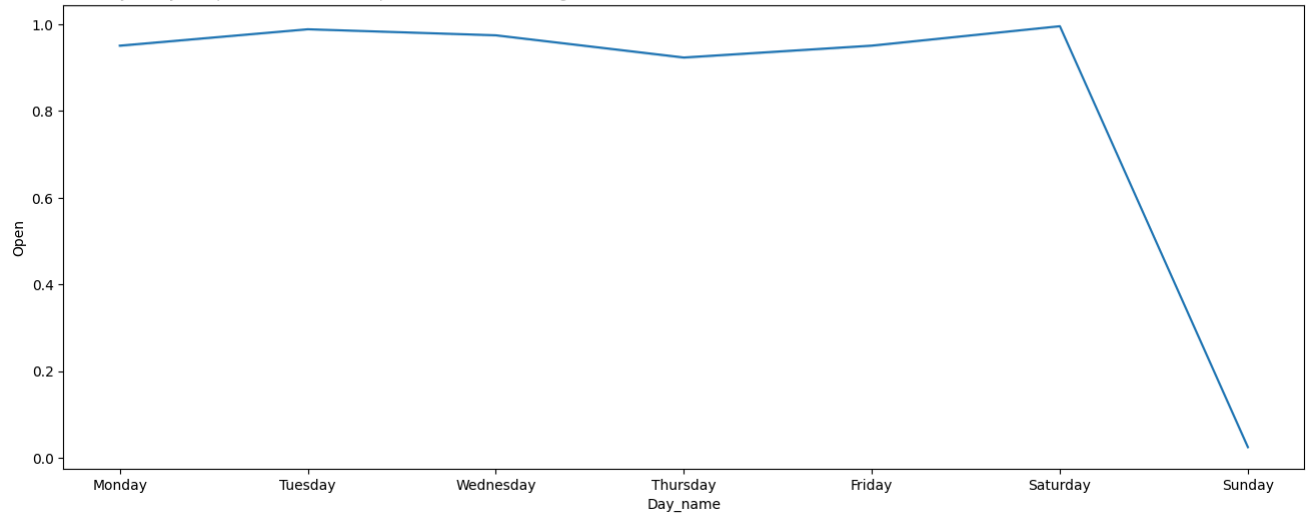


Context: Rossmann is a German drugstore chain. In Germany, stores are legally required to close on Sundays

```
plt.figure(figsize=(16,6))
sns.lineplot(data=df.sort_values(by='DayOfWeek'),x='Day_name',y='Open')
plt.suptitle('Open PCT% By Day', fontweight='bold', x=0.13, fontsize=12,ha='left')
plt.title('Sunday is only 2% open of all times, this explains the low sales average', x=0.01, fontsize=10,ha='left')
plt.show()
```

Open PCT% By Day

Sunday is only 2% open of all times, this explains the low sales average



Ghost Store

Open But 0 Sales Stores

```
ghost = df[(df['Open']==1) & (df['Sales']==0)]
ghost.head()
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	Day_name	Month_name	Month	Year
86825	971	5	2015-05-15	0	0	1	0	0	1	Friday	May	5	2015
142278	674	4	2015-03-26	0	0	1	0	0	0	Thursday	March	3	2015
196938	699	4	2015-02-05	0	0	1	1	0	0	Thursday	February	2	2015

Next steps: [Generate code with ghost](#) [New interactive sheet](#)

These are likely data errors (server outages, inventory audits) or "closed for refurbishment" days that weren't logged correctly.

```
clean_df = df[(df['Open']==0) | (df['Sales']>0)]
clean_df.sample(10)
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday	Day_name	Month_name	Month	Year
	4418	1074	2	2015-07-28	7110	585	1	1	0	1	Tuesday	July	7 2015
	128725	501	2	2015-04-07	4694	571	1	0	0	1	Tuesday	April	4 2015
	554839	355	3	2014-02-19	8199	979	1	1	0	0	Wednesday	February	2 2014
	661388	979	6	2013-11-16	5266	589	1	0	0	0	Saturday	November	11 2013
	46142	428	6	2015-06-20	5973	513	1	0	0	0	Saturday	June	6 2015
	977881	812	2	2013-02-05	8871	1033	1	1	0	1	Tuesday	February	2 2013
	554839	355	3	2015-02-19	8199	979	1	1	0	0	Wednesday	February	2 2015

```
print("Full Dataset: ", df.shape[0])
print("Ghost Stores: ", ghost.shape[0])
print("After Cleaning: ", clean_df.shape[0])
```

```
Full Dataset: 1017209
Ghost Stores: 54
After Cleaning: 1017155
```

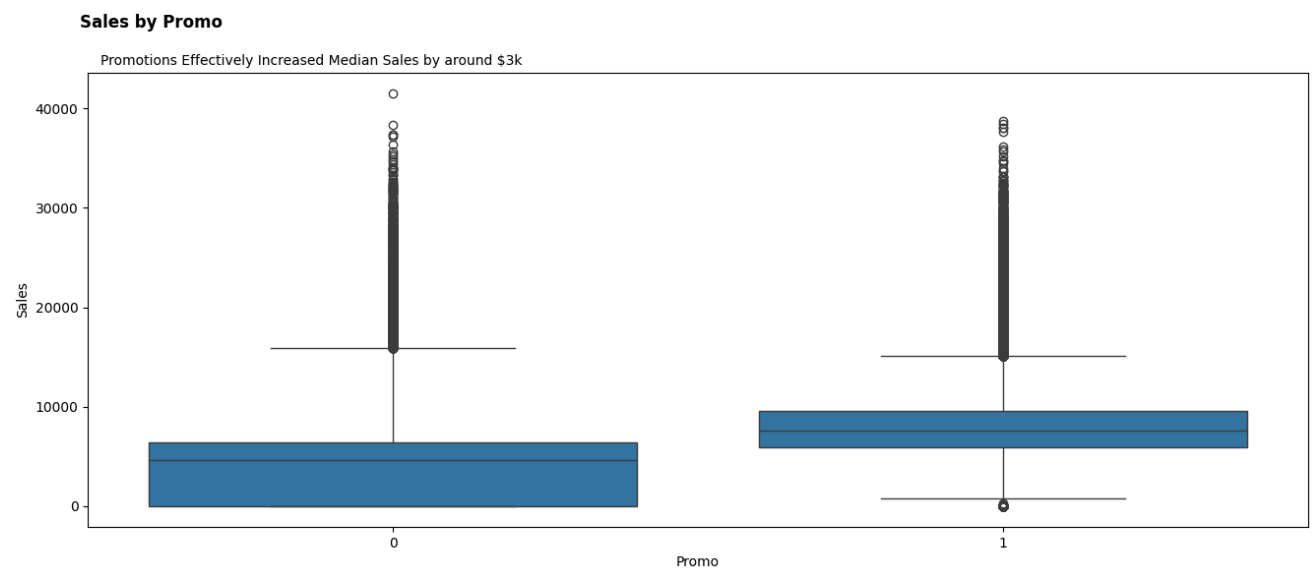
▼ Promo Lift

```
promo = clean_df.groupby('Promo')['Sales'].agg(['mean', 'min', 'median', 'max', 'count', 'std']).reset_index()
print(promo)
```

```

Promo      mean  min  median  max  count      std
0         0  4406.281930    0  4622.0  41551  629096  3442.392390
1         1  7991.584491    0  7554.0  38722  388059  3418.612585
```

```
plt.figure(figsize=(16,6))
sns.boxplot(data=clean_df,x='Promo',y='Sales')
plt.suptitle('Sales by Promo', fontweight='bold', x=0.12, fontsize=12, ha='left')
plt.title('Promotions Effectively Increased Median Sales by around $3k', x=0.01, fontsize=10,ha='left')
plt.show()
```



Promotions Effectively Increased Median Sales by around \$3000

✓ The Forecast

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

✓ Feature Engineering:

Removing Object Data For ML

```
ml_df = clean_df.copy()
ml_df.drop(columns=['Month_name', 'Day_name'], inplace=True)
ml_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1017155 entries, 0 to 1017208
Data columns (total 11 columns):
 #   Column          Non-Null Count  Dtype
---  -
 0   Store           1017155 non-null  int64
 1   DayOfWeek       1017155 non-null  int64
 2   Date            1017155 non-null  datetime64[ns]
 3   Sales           1017155 non-null  int64
 4   Customers       1017155 non-null  int64
 5   Open            1017155 non-null  int64
 6   Promo           1017155 non-null  int64
 7   StateHoliday    1017155 non-null  int64
 8   SchoolHoliday   1017155 non-null  int64
 9   Month           1017155 non-null  int32
10  Year            1017155 non-null  int32
dtypes: datetime64[ns](1), int32(2), int64(8)
memory usage: 85.4 MB
```

Selecting Store

```
ml_df=ml_df[ml_df['Store']==1]
```

Train, Test Datasets

```
train_df = ml_df[ml_df['Date'] < '2015-06-14']
test_df = ml_df[ml_df['Date'] > '2015-06-14']

X_train = train_df.drop(columns=['Date', 'Sales', 'Customers'])
y_train = train_df['Sales']

X_test = test_df.drop(columns=['Date', 'Sales', 'Customers'])
y_test = test_df['Sales']
```

✓ Modeling

Fitting

```
rf=RandomForestRegressor()
rf.fit(X_train,y_train)
```

```
▼ RandomForestRegressor ⓘ ?
RandomForestRegressor()
```

Predicting

```
y_pred= rf.predict(X_test)
```

Scoring

```
print(f"R2 Score: {r2_score(y_test,y_pred)}")
print(f"Mean Absolute Error: {mean_absolute_error(y_test,y_pred)}")
print(f"Mean Squared Error: {mean_squared_error(y_test,y_pred)}")
print(f"Root Mean Squared Error: {np.sqrt(mean_squared_error(y_test,y_pred))}")
```

```
R2 Score: 0.9283309236984885
Mean Absolute Error: 306.07619056829714
Mean Squared Error: 187872.83670325752
Root Mean Squared Error: 433.4430028311191
```

Comaprisn

```
test_df['Predicted_Sales']=y_pred
```

```
plt.figure(figsize=(16,6))
sns.lineplot(data=test_df,x='Date',y='Sales')
sns.lineplot(data=test_df,x='Date',y='Predicted_Sales')
plt.suptitle('Actual vs Predicted Sales', fontweight='bold', x=0.125, fontsize=12, ha='left')
plt.title('Model scored 92.8% accuracy for the last 6-weeks Sales Prediction of store 1', x=0.01, fontsize=10, ha='left')
plt.legend(['Actual', '', 'Predicted'])
plt.show()
```

